

WEB APPLICATION PROGRAMMING

By Ukesh Thapa

Course Structure

- ❖ Introduction (6 hours)
- ❖ JavaScript and Client-Side Programming (12 hours)
- ❖ Server-Side Web Programming (9 hours)
- ❖ Web Services and APIs (7 hours)
- ❖ Web Application Security (6 hours)
- ❖ Web Application Deployment and Modern Trends (5 hours)

Marks Distribution

Chapter	Hours	Marks distribution*
1	6	8
2	12	16
3	9	12
4	7	9
5	6	8
6	5	7
Total	45	60

The Internet

- The global network of interconnected computers that communicate with each other using standard rules (**protocols**).
- The word “Internet” is a contraction of “internetwork” (i.e. a network of networks).
- The first occurrence of “Internet” (in that sense) appears in **1974**.
- By Cerf (and colleagues).



Fig1. Submarine Cable Map

<https://www.submarinecablemap.com/>

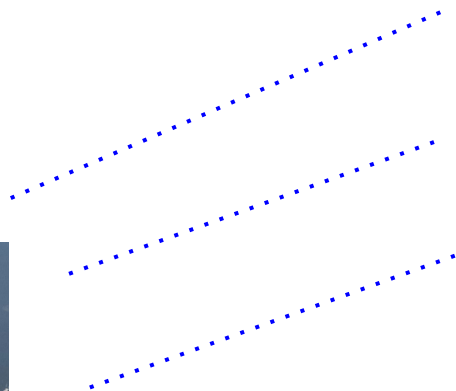
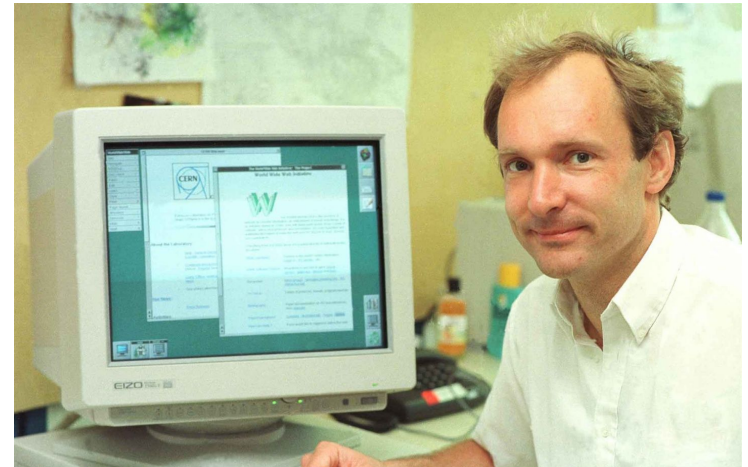
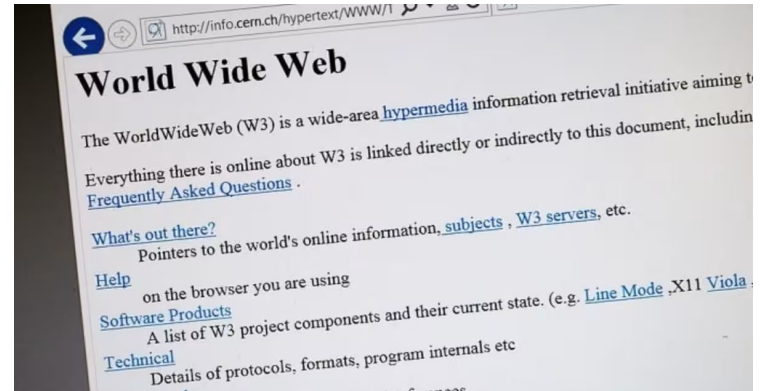


Fig 2. Starlink

The Web

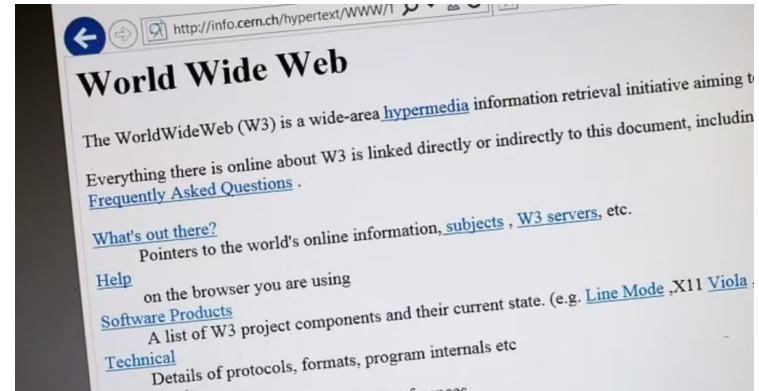
- The Web is a software system running on top of the Internet hardware.
- The Web is a system of connected documents and resources that can be viewed and interacted with through a browser, using technologies like HTML, HTTP, and URLs.
- The term “**World Wide Web (WWW)**” was coined by **Sir Tim Berners-Lee**, a British scientist at CERN.



Tradition Web Applications

- **Server-Side Rendering (SSR)**
 - HTML pages are generated on the server.
 - Browser receives a complete page every time.
- **Full Page Reloads**
 - Each click → server processes → returns a new webpage.
 - No smooth transitions or dynamic updates.
- **Backend-Heavy Architecture**
 - Business logic, data processing, and routing are handled on the server.
- **Limited Interactivity**
 - Minimal JavaScript.
 - Very few dynamic features.
 - Applications feel slower and less responsive compared to modern web apps.

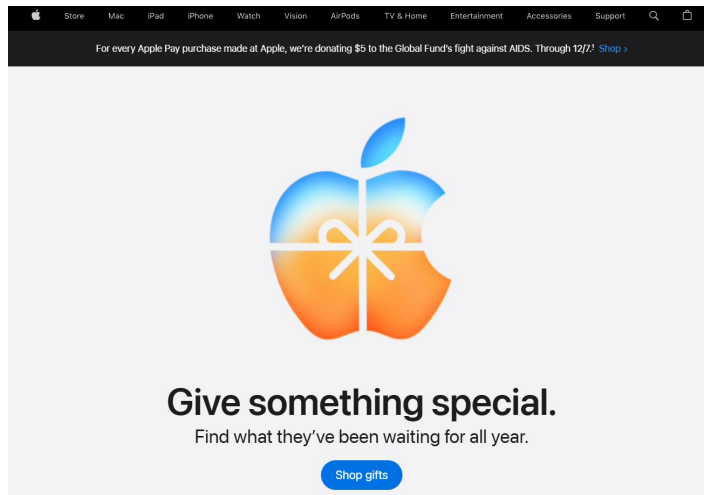
For Example:



https://en.wikipedia.org/wiki/World_Wide_Web

Modern Web Applications

- **Client-Side Rendering (CSR) + Hybrid Rendering**
 - Most HTML is generated inside the browser using JavaScript.
 - Initial page loads once; after that only data changes.
 - Modern frameworks like React, Angular, Vue update UI dynamically.
- **No Full Page Reloads**
 - Page stays loaded.
 - Only small pieces of data are fetched from APIs (JSON).
 - Smooth transitions, animations, and app-like behavior.



Modern Web Applications

- **Frontend-Heavy Architecture**

- Browser handles UI rendering, routing, and state management.
- Server mostly provides APIs for data.
- Separation of frontend (React/Vue) and backend (Node/Django/FastAPI).

- **High Interactivity**

- Rich JavaScript.
- Dynamic components update instantly without reloads.
- Real-time features using WebSockets, live updates, notifications.

For Example:

<https://www.youtube.com/>

<https://www.samsung.com/us/>

History of Web

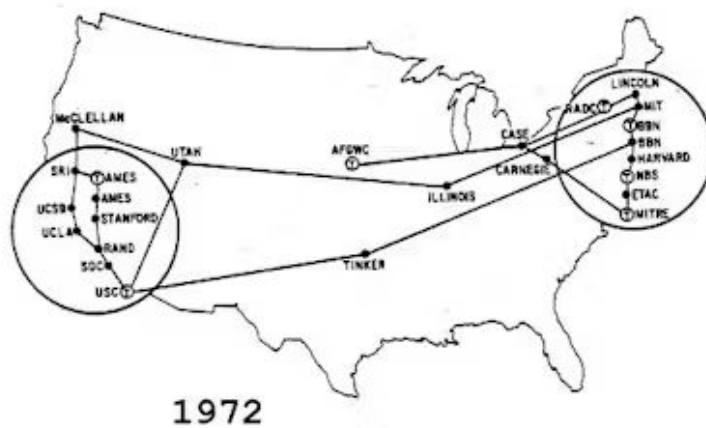
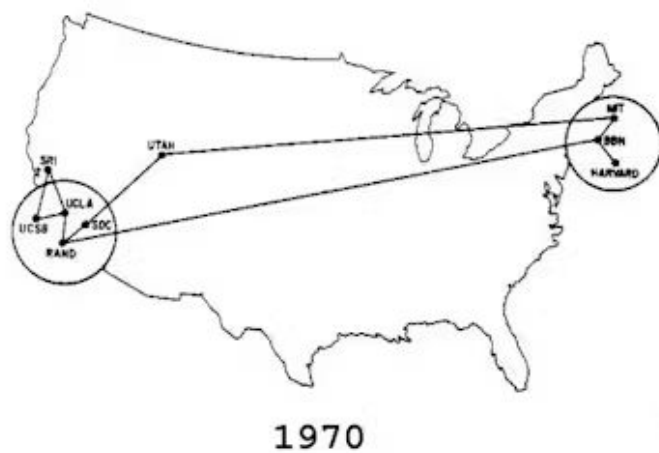
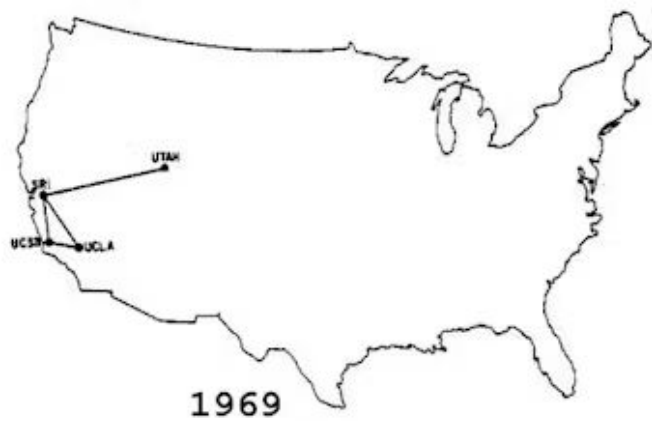
❖ Late 1960:

- The Department of Defense (**DoD**), specifically through its research arm **ARPA** (Advanced Research Projects Agency), took the critical step of funding and building the first computer network, which became the blueprint for the modern internet

❖ 1969: ARPANET

- Concept: "Packet Switching", the first message ("LO") traveled from UCLA to Stanford.
- There were no "websites." Data was raw code or text messages sent between terminals. You had to be a computer scientist to use i

<https://www.youtube.com/shorts/wUhBkcHKz6E>



History of Web

❖ The Static Era: The "File" Paradigm (1990–1995):

- A "**web page**" was exactly that—a page.
- It was a physical file sitting on a computer.
- Mechanism: The user asks for file.html. The server finds file.html on its hard drive and sends it. There is no processing, no database, and no logic.
- **1990:** Tim Berners-Lee creates the first web server and browser.
 - info.cern.ch the every first website
- **1993: Mosaic Browser** is released. It can display images inline, popularizing the static web.
- **1994: CSS (Cascading Style Sheets)** is proposed. Before this, design was hard-coded into the HTML structure.

History of Web

❖ The Early Dynamic Era: The "Server-Side" Paradigm (1995–2004)

- Mechanism: The user asks for a page. The server runs a script (code), fetches data from a database (SQL), "glues" it into an HTML template, and sends the finished page to the user.
- **1993/1994:** CGI (Common Gateway Interface) is standard. This was the first technology that allowed web servers to execute external programs (scripts) to generate content dynamically.
- **1994: Yahoo** was developed due to the large number web site. Introduce as the search engine https://www.youtube.com/watch?v=KZeIEiBrT_w,
- **1995:** PHP is released. It made server-side scripting accessible, allowing code to be embedded directly into HTML.
- **1995:** JavaScript is invented (by Netscape). Initially, it was only used for simple "dynamic" visual tricks like pop-up alerts or scrolling text, not data.

History of Web

- **1996:** ASP (Active Server Pages) released by Microsoft. This brought dynamic server capability to enterprise environments.
- **1998:** Mozilla is formed, leading to the eventual rise of Firefox and open web standards. And same **Era Google** was founded
- ❖ **The Web 2.0 Era: The "Asynchronous" Paradigm (2004–2010)**
 - Every time you clicked a button, the entire screen went white and reloaded.
 - **Mechanism (AJAX):** The browser could send a request to the server in the background (using JavaScript) and update just one part of the screen without reloading the whole page.
 - **2004:** Gmail launches. It was the first major app to feel like a desktop program because it didn't reload when you opened an email.

History of Web

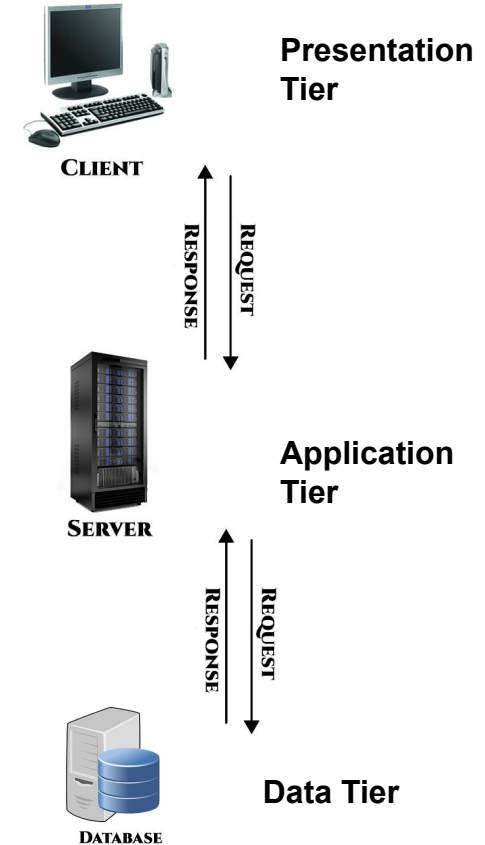
- **2005:** The term AJAX (Asynchronous JavaScript and XML) is coined to describe this new technique of background data fetching.
- **2006:** jQuery is released. It simplified JavaScript so much that almost every website began adding interactive, dynamic elements.
- **2008:** Google Chrome is released with the V8 engine, making JavaScript fast enough to run complex applications, not just simple scripts.
- ❖ **The Modern Era: The "Single Page Application" (SPA) Paradigm (2010–Present)**
 - The web effectively detached the "front end" (what you see) from the "back end" (the data).
 - The website is now a standalone application that lives in your browser.
 - Mechanism: When you visit a site, the server sends you a blank "shell" and a large bundle of JavaScript code. Your browser runs that code to draw the interface. It only asks the server for raw data (JSON), not HTML pages.

History of Web

- **2010:** AngularJS (by Google) is released. It popularized the "Single Page Application" (SPA) model.
- **2013:** React (by Facebook) is released. It introduced the "Component" model, where parts of the UI (like a chat box) manage their own state independently.
- **2014:** HTML5 is officially recommended. It gave browsers native powers previously reserved for Flash (playing video, drawing graphics, saving files).
- **2015:** Progressive Web Apps (PWA) terminology emerges. Web apps can now work offline and send push notifications, behaving almost exactly like native mobile apps.

Multi-tier Architecture

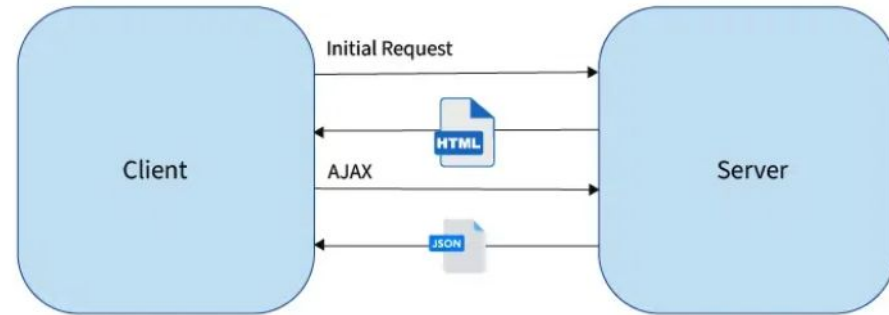
- ❖ This also known as **n-tier** or **Three-tier architecture** is a client-server architecture.
- ❖ It is separated as **presentation**, **application processing** and **data management**.
- ❖ **Presentation Tier:**
 - This is the topmost level of the application.
 - Displays information related to such services as browsing merchandise
 - It is a layer that users can access directly
- ❖ **Application Tier:**
 - The logical tier is pulled out from the presentation tier and, as its layer, controls an application's functionality by performing detailed processing.
- ❖ **Data Tier:**
 - The bottom-most layer responsible for persistent storage and retrieval of the application's data.
 - Managing access rights and ensuring the physical security of the data itself.
 - Database systems like MySQL, PostgreSQL, Oracle, and others



Single Page Architecture

- ❖ It loads and updates content dynamically without refreshing the entire page.
- ❖ It minimizes interruptions and providing a smoother interface.
- ❖ When to use: **Dynamic Interactions, Real-Time Updates, Rich User Interfaces**
- ❖ How Does this work?
 - **Initial Load:** The user navigates to the URL for the first time. The server sends a single HTML file along with CSS and the JavaScript.
 - **User Interaction:** User perform action and Javascript handle events, preventing from requesting a new HTML page
 - **API Request:** JS sends an asynchronous background request to fetch necessary data only

- **Client-Side Rendering:** The SPA's framework receives the JSON data. The JavaScript uses this data to dynamically generate the new HTML elements
- **Managing Browser History:** To ensure the back/forward buttons still work, the SPA uses the History API to update the browser's URL and history state without causing a page refresh.

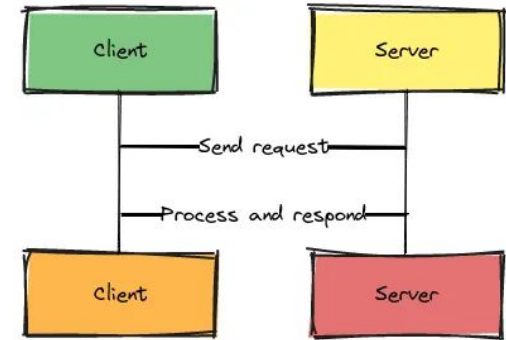


Client Server Architecture

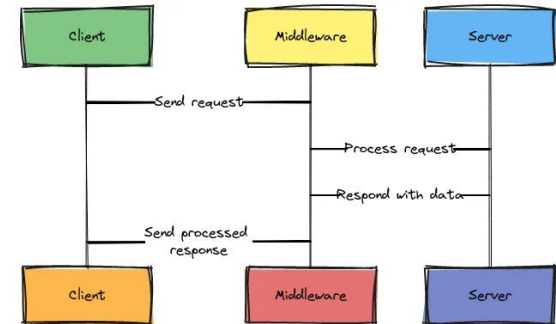
- ❖ **Client-server architecture** is a **network model** in which two main entities — **clients** and **servers** — communicate with each other to complete specific tasks or share data.
- ❖ The client-server model consists of two main components:
 - **Client:** The client initiates requests, waits for the server's response, and displays it to the user.
 - **Server:** The server processes these requests, retrieves the relevant information, and sends it back to the client.

Types:

2 Tier:



3 Tier:



Client-Server Architecture

❖ Advantages:

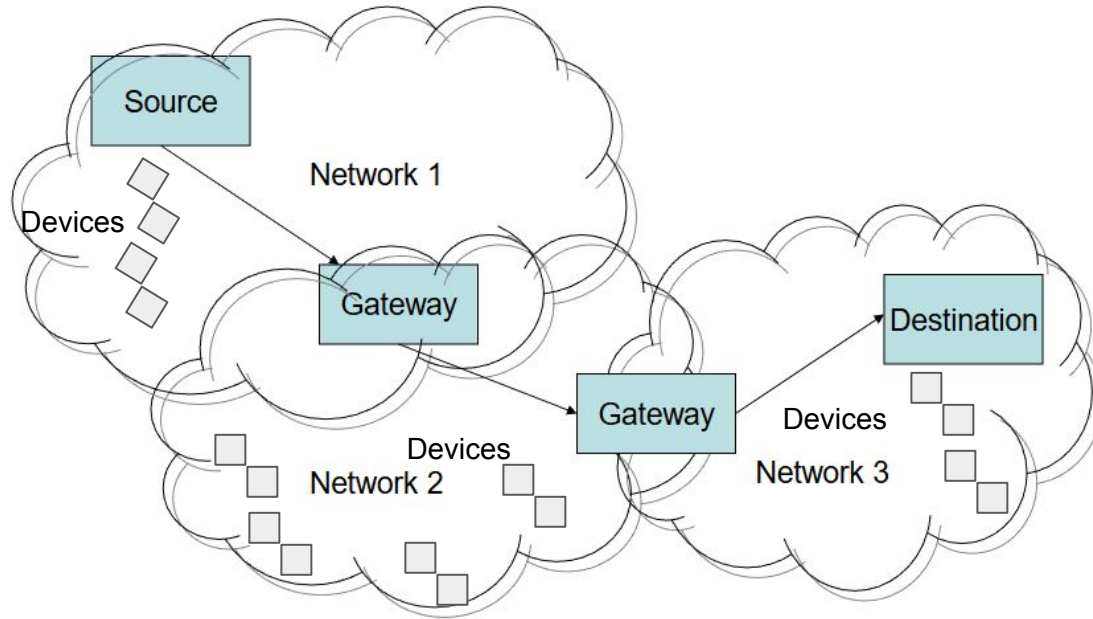
- **Centralized Management & Security:** The server acts as a single point of control for managing data storage, security policies, and software updates.
- **Enhanced Scalability:** The architecture allows for independent scaling. You can upgrade the server (to handle more processing) or add more clients (users) without having to modify the other component, ensuring the system can grow easily.
- **Efficient Resource Sharing:** It enables multiple clients to share expensive, centralized resources like high-capacity printers, large databases, and specialized software licenses, reducing overall costs and redundancy.
- **Foundation for Networked Services:** This model is necessary for any application requiring multiple users to access shared, consistent data over a network.

Internet Protocol (IP)

- ❖ Communication protocol: how computers talk
- ❖ IP developed as part of ARPANET research
 - ARPANET began using **TCP/IP** in 1982
- ❖ Designed for use both within **local area networks** (LAN's) and between networks\
- ❖ **IP Address:** A numerical label assigned to devices on a computer network for communication. It has **32 bit** number (IPV4)
- ❖ IP function: transfer data from source device to destination device
- ❖ IP source software creates a packet representing the data
 - Header: source and destination IP addresses, length of data, etc.
 - Data itself
- ❖ If destination is on another LAN, packet is sent to a gateway that connects to more than one network

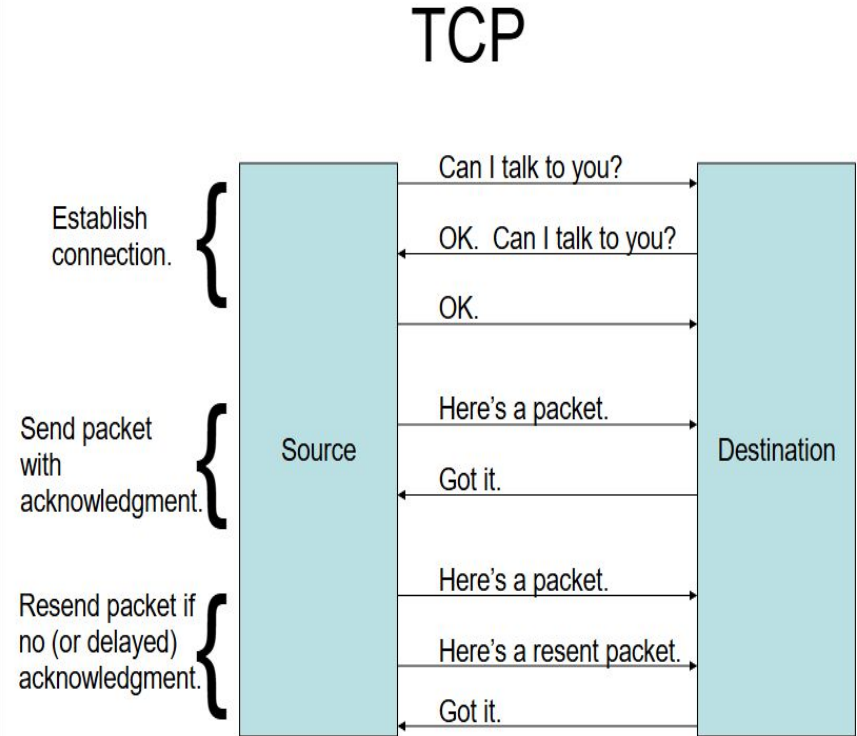
Internet Protocol (IP)

IP



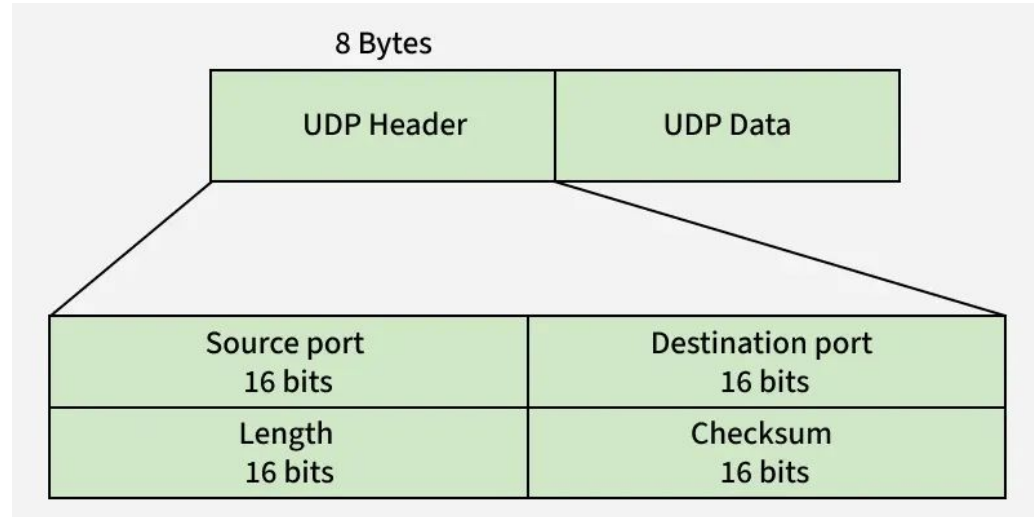
Transmission Control Protocol

- ❖ Limitations of IP:
 - No guarantee of packet delivery (packets can be dropped)
 - Communication is one-way (source to destination)
- ❖ **TCP** adds concept of a **connection on top** of IP: Provides guarantee that packets delivered, Provide two-way (full duplex) communication
- ❖ TCP also adds concept of a port
 - TCP header contains port number representing an application program on the destination compute
 - Example: port 25 is normally used for email transmitted using the Simple Mail Transfer Protocol (SMTP)



User Datagram Protocol (UDP)

- ❖ Like TCP in that:
 - Builds on IP
 - Provides port concept
- ❖ Unlike TCP in that:
 - No connection concept
 - No transmission guarantee
- ❖ Advantage of UDP vs. TCP:
 - Lightweight, so faster for one-time messages
- ❖ **Real life:** Live streaming (Twitch, YouTube Live), online gaming, VoIP/video calls (Zoom, Skype), DNS (fast domain name lookups), DHCP, and IoT



World Wide Web (WWW)

- ❖ Originally, one of several systems for organizing Internet-based information
 - Competitors: WAIS (keyword based search), Gopher, ARCHIE (FTP sites for file searches)
- ❖ Distinctive feature of Web: support for hypertext (text containing **links**)
 - Communication via Hypertext Transport Protocol (HTTP)
 - Document representation using Hypertext Markup Language (HTML)
- ❖ The Web is the collection of machines (Web servers) on the Internet that provide information, particularly HTML documents, via HTTP.
- ❖ Machines that access information on the Web are known as Web clients.
- ❖ A Web browser is software used by an end user to access the Web.

Hypertext Transport Protocol (HTTP)

- ❖ HTTP is based on the request-response communication model:
 - Client sends a request
 - Server sends a response
- ❖ HTTP is a stateless protocol
- ❖ **80** is standard port number for HTTP in TCP IP
- ❖ The information transmitted using HTTP is often entirely text
- ❖ Can use the Internet's Telnet protocol to simulate browser request and view server response

```
bosko@pnep:~$ telnet google.com 80
Trying 142.251.39.78...
Connected to google.com.
Escape character is '^]'.
GET /index.html
HTTP/1.0 200 OK
Date: Mon, 23 Jan 2023 20:38:19 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
Cross-Origin-Opener-Policy-Report-Only: same-origin-allow-popups; report-to="gws"
Report-To: {"group":"gws","max_age":2592000,"endpoints":[{"url":"https://csp.wit
hgoogle.com/csp/report-to/gws/other"}]}
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2023-01-23-20; expires=Wed, 22-Feb-2023 20:38:19 GMT; path=/;
```

HTTP Request and Response

```
Connect { $ telnet www.example.org 80
        Trying 192.0.34.166...
        Connected to www.example.com
        (192.0.34.166).
        Escape character is '^]'.
Send    { GET / HTTP/1.1
Request { Host: www.example.org

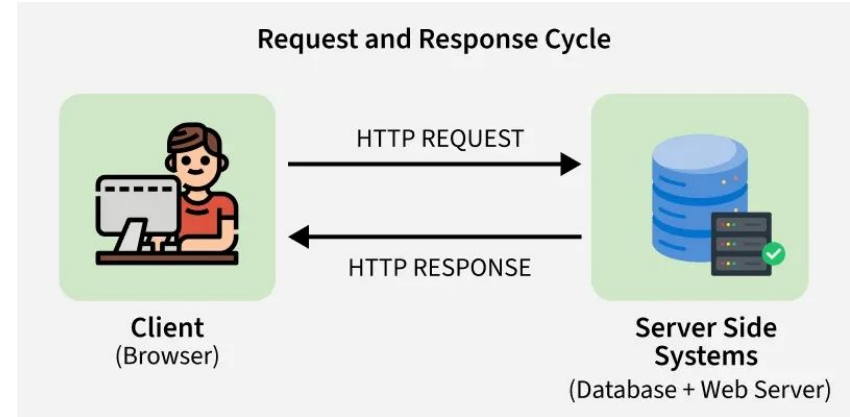
Receive { HTTP/1.1 200 OK
Response { Date: Thu, 09 Oct 2003 20:30:49 GMT
          ...
```

HTTP STATUS CODE

Class	Class Range	Meaning	Client Action
1xx	100-199	Informational	Request received and processing continues.
2xx	200-299	Success	The request was successfully received, understood, and accepted.
3xx	300-399	Redirection	Further action needs to be taken to complete the request (often involving a new URL).
4xx	400-499	Client Error	The request contains bad syntax or cannot be fulfilled by the server.
5xx	500-599	Server Error	The server failed to fulfill an apparently valid request.

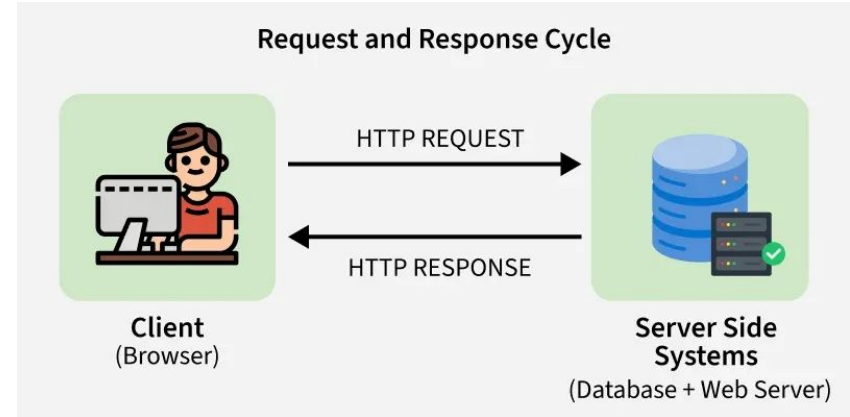
HTTP Request and Response Cycle

- ❖ The HTTP Request–Response Cycle is the fundamental communication mechanism that allows web clients (like your browser) to retrieve resources or submit data to web servers.
- ❖ The five main stages:
 - **Connection Establishment (TCP Handshake):**
 - Before sending the HTTP request, the client must first connect to the server.
 - The client uses the IP address and Port (e.g., 80 for HTTP, 443 for HTTPS), obtained via the DNS lookup, to locate the server.
 - The client and server perform a TCP three-way handshake (SYN, SYN-ACK, ACK) to establish a reliable, ordered connection channel.



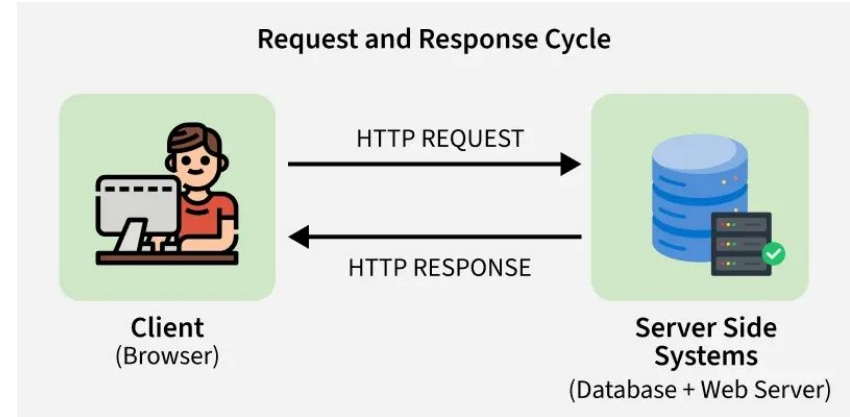
HTTP Request and Response Cycle

- **Client Sends the HTTP Request:** Once the connection is secure and established, the client transmits its request to the server.
 - Request Line: Specifies the HTTP Method (e.g., GET, POST, PUT), the path to the resource (e.g., /index.html), and the HTTP version.
 - Headers: Key-value pairs providing metadata, such as the Host address, User-Agent (client info), and Accept (what file types the client can handle).
 - Body (Optional): Used primarily with POST or PUT methods to send data (e.g., form data, JSON, XML) to the server.
- **Server Processing and Logic:**
 - The server's software (like Apache, Nginx, or a Node.js application) reads the request and headers.
 - It executes the business logic associated with the requested



HTTP Request and Response Cycle

- **Server Sends the HTTP Response:** After processing the request, the server prepares and sends a response back to the client.
 - Status Line: Includes the HTTP version and a Status Code (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
 - Headers: Metadata about the response, such as the Content-Type (e.g., text/html, application/json), Content-Length, and caching instructions.
 - Body: The actual requested data (e.g., the HTML code for a webpage, an image file, or a JSON object).
- **Client Processing and Rendering:** The client receives the response and takes action based on the Status Code and Content-Type.
 - All the necessary information (HTML, CSS, and JS code) are render.
 - Finally, the connection may be closed, or it may be kept open for future requests (a process called Keep-Alive) to improve efficiency.



Hypertext Transfer Protocol Secure (HTTPS)

- ❖ HTTPS is a secure communication protocol used for transmitting data over the internet.
- ❖ It ensures that data exchanged between a user's web browser and a website's server is encrypted and secure.
- ❖ It uses the same application-layer protocol as HTTP, but encrypts it using the TLS/SSL layer before it is transported over the network.
- ❖ **Main Reason:** The Need for Security and Privacy
- ❖ In 1994 for their browser to secure transactions for the rapidly growing e-commerce sector. The main catalyst was securing data like credit card numbers and passwords.
- ❖ Plain HTTP sends data in plaintext (unencrypted), meaning anyone sniffing the network traffic (e.g., on public Wi-Fi) can easily read the data. This vulnerability, known as the "Man-in-the-Middle" (MITM) attack, was the fundamental problem SSL/TLS was designed to solve.

Benefits & Importance of HTTPS

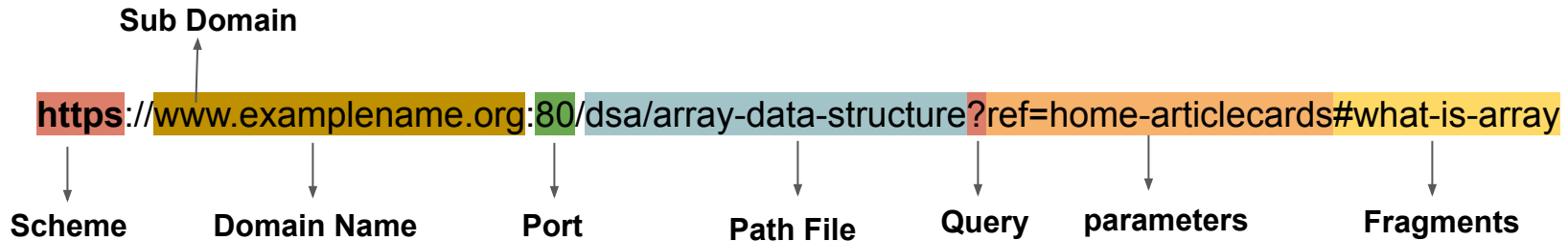
- ❖ The adoption of HTTPS is driven by three essential security pillars: Encryption, Integrity, and Authentication.
 - **Encryption** (Confidentiality)
 - HTTPS scrambles the data exchanged between the client and the server.
 - If an attacker intercepts the data, they will only see unreadable ciphertext. This prevents eavesdropping and protects sensitive information like usernames, passwords, and personal details.
 - **Data Integrity**
 - TLS/SSL uses cryptographic hashing to create a digital fingerprint (a message authentication code) of the data being sent.
 - **Authentication** (Trust)
 - HTTPS relies on TLS/SSL Certificates issued by trusted third-party Certificate Authorities (CAs). The server presents this certificate during the initial connection.

Uniform Resource Identifier (URI)

- ❖ The URI concept was developed in the early 1990s by Tim Berners-Lee
- ❖ A URI (Uniform Resource Identifier) was created to provide a single, unified standard for addressing and identifying any resource on the network, regardless of whether that resource is a webpage, an email address, a book, or a physical item.
- ❖ Example:
 - Full URI = <http://www.example.com/articles/latest.html?id=101>
 - Scheme = Http
 - Host Header = www.example.com
 - Request-URI = /articles/latest.html?id=101 , where “/” this is root directory
- ❖ **Types:**
 - URL (Uniform Resource Locator)
 - URN (Uniform Resource Name)

Uniform Resource Locator (URL)

- ❖ A Unique identifier that is contained by all the resources available on the internet.
- ❖ It can help to locate a particular resource due to its uniqueness.
- ❖ It is also known as the web address.
- ❖ The users can access the URLs by simply typing them inside the address bar or by clicking any button or link web page. For example: <https://www.google.com/>
- ❖ It uses multiple protocols like HTTP (Hypertext Transfer Protocol), HTTPS Protocol (Secured HTTP), mailto for emails, FTP (File Transfer Protocol) for files, and TELNET to access remote computers.



Domain Name System (DNS)

- ❖ DNS is the “phone book” for the Internet
 - Map between host names and IP addresses
 - DNS often uses UDP for communication
- ❖ Its primary function is to translate human-readable domain names (like www.google.com) into computer-readable IP addresses (like 142.250.66.100).
- ❖ Host names
 - Labels separated by dots, e.g., www.example.org
 - Final label is top-level domain Generic: .com, .org, etc. Country-code: .us, .il, etc.
- ❖ **DNS Lookup:**
 - DNS Lookup, also called DNS Resolution, is the process of translating a human-readable domain name into its corresponding IP address.
 - **DNS Resolver:** Initiates the lookup process. Also called a DNS client.
 - **Recursive Query:** Resolver queries multiple servers on behalf of the client until the IP is found.
 - **Iterative Query:** Resolver asks servers for the best answer available.
 - **Non-Recursive Query:** Resolver queries a server that already has the record in its cache.

Domain Name System (DNS)

Step 1: The Client Query

Your Browser first checks its own cache (local memory) to see if it has the IP address for example.com already. If not, it sends a query to your local DNS Resolver.

Step 2: The Resolver's Check

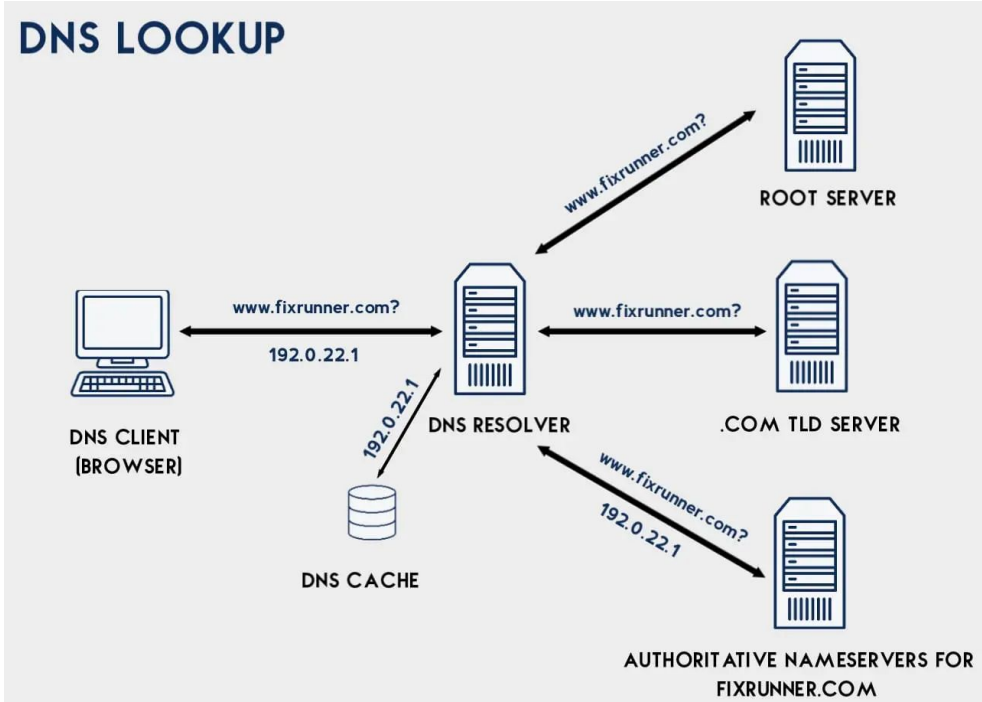
The Resolver checks its own large cache. If the address is found, the lookup stops here, and the IP is immediately sent back to the browser. (This is the fastest outcome!)

Step 3: Asking the Root

If the address isn't in the cache, the Resolver asks the Root Name Server for the address of example.com.

Step 4: Referral to the TLD

The Root Name Server sees the .com part and replies to the Resolver, saying: "I don't know the IP, but I know the address for the .com TLD Server. Go ask them."



Domain Name System (DNS)

Step 5: Asking the TLD

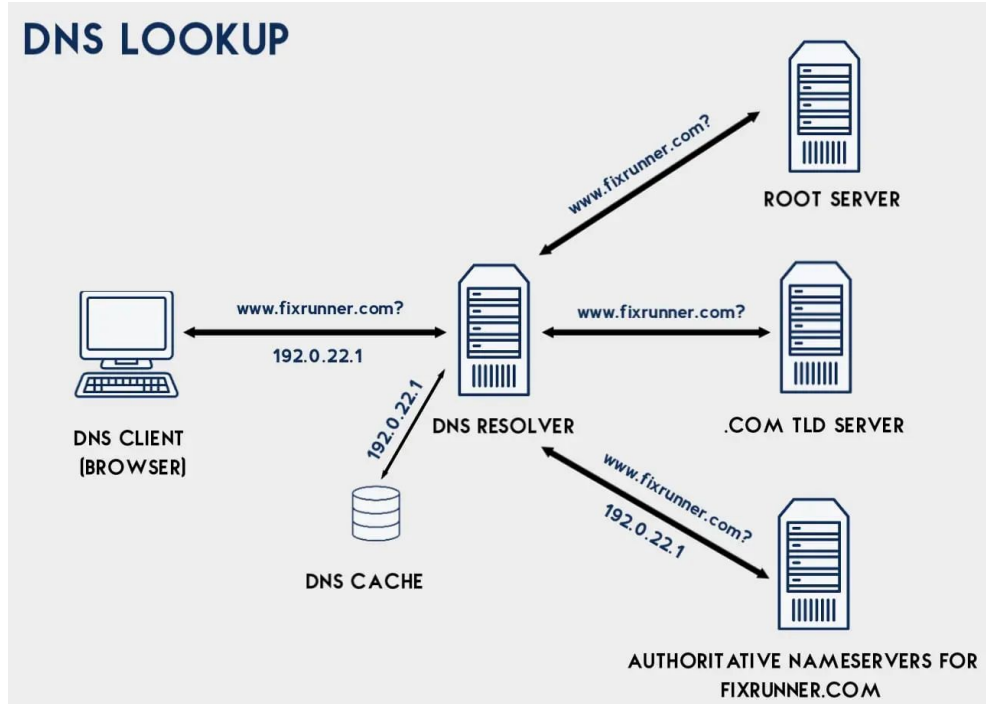
The Resolver then asks the TLD Name Server (the server managing all .com domains). The TLD server sees the example.com part and replies: "I don't know the IP, but I know the address of the Authoritative Server for example.com. Go ask them."

Step 6: Asking the Authoritative Server

The Resolver asks the Authoritative Name Server for example.com. This server holds the true record and finally replies with the correct IP address (e.g., 192.0.2.1).

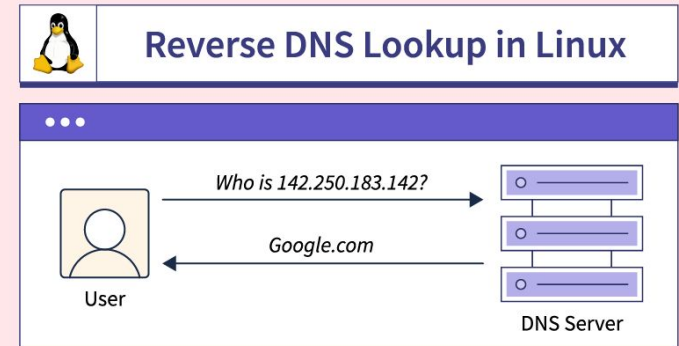
Step 7: Final Delivery and Caching

The Resolver sends the correct IP address back to the Browser. Critically, the Resolver also caches the IP address so it can answer immediately the next time someone asks for example.com (returning to Step 2).



Domain Name System (DNS)

- ❖ **DNS caching** : A mechanism that stores DNS records locally to avoid querying external DNS servers repeatedly for the same information. This speeds up the browsing experience and reduces network traffic.
- ❖ **TTL (Time-to-Live)** is the amount of time that a DNS record is cached before it expires. When the TTL expires, the cache is cleared and a fresh DNS query must be made. The TTL is defined by the authoritative DNS server, which can adjust this based on the nature of the domain.
- ❖ **Reverse DNS Lookup**
 - Reverse DNS Lookup is the process of mapping an IP address back to its corresponding domain name.
 - This is the opposite of the typical DNS lookup, where a domain name is resolved to an IP address.
- ❖ **Use Cases**
 - Anti-Spam Filtering
 - Security & Logging
 - Verification

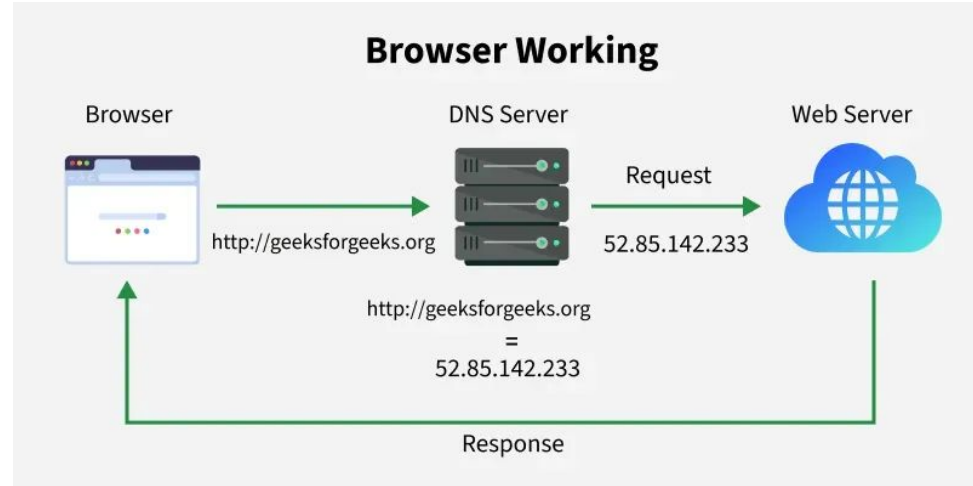


Web Browsers

- ❖ The web browser is an application software used to explore the World Wide Web (WWW).
- ❖ Primary tasks:
 - Convert web addresses (URL's) to HTTP requests
 - Communicate with web servers via HTTP
 - Render (appropriately display) documents returned by a server
- ❖ Additional functionality:
 - Execution of scripts (e.g., drop-down menus)
 - Event handling (e.g., mouse clicks)
 - GUI for controls (e.g., buttons)
 - Secure communication with servers
 - Display of non-HTML documents (e.g., PDF) via plug-ins

Web Browsers

- ❖ How Does a Web Browser Work?
 - **Phase 1: Requesting the Resource**
 - Handling the URL
 - DNS Lookup
 - Establishing the Connection via TCP handshake
 - Sending the HTTP Request
 - **Phase 2: Processing the Response**
 - Receiving the Data
 - Building the DOM and CSSOM
 - **Phase 3: Rendering the Page**
 - The Render DOM Tree and CSSOM Tree
 - Render the HTML and CSS
 - Executing JavaScript



Hyper Text Markup Language (HTML)

- It was published in 1995
- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML elements tell the browser how to display the content

HTML Elements

An HTML element is defined by a start tag, some content, and an end tag

`<tagname>` Content goes here `</tagname>`

For example: `<p>`, `<h1>`, and others

HTML

Declaration defines that this document is an HTML5 document

The root element of an HTML page

Meta information about the HTML page

Element specifies a title for the HTML page

Element defines the document's body, and is a container for all the visible contents

Elements for the paragraph and heading

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Page Title</title>
```

```
</head>
```

```
<body>
```

```
<h1>My First Heading</h1>
```

```
<p>My first paragraph.</p>
```

```
</body>
```

```
</html>
```

HTML Elements

❖ HTML Headings

- `<h1>Heading 1</h1>`
- `<h2>Heading 2</h2>`
- `<h3>Heading 3</h3>`
- `<h4>Heading 4</h4>`
- `<h5>Heading 5</h5>`
- `<h6>Heading 6</h6>`

❖ HTML Paragraph

- `<p>This is a paragraph.</p>`

❖ HTML Link

- `<a href="url">link text</a>`
- Attributes
 - `_self` - Default. Opens the document in the same window/tab as it was clicked
 - `_blank` - Opens the document in a new window or tab
 - `_parent` - Opens the document in the parent frame
 - `_top` - Opens the document in the full body of the window

HTML Elements

❖ HTML Photos

➤ ``

❖ HTML Button

➤ `<button>Click Me</button>`

❖ Others Tags

Tags	Description
<code></code>	Defines bold text
<code>
</code>	Defines a single line break
<code><canvas></code>	Used to draw graphics, on the fly, via scripting (usually JavaScript)
<code><div></code>	Defines a section in a document
<code><script></code>	Defines a client-side script
<code></code>	Defines important text
<code><style></code>	Defines style information for a document
<code><sub></code>	Defines subscripted text
<code></code>	Defines an unordered list
<code></code>	Defines an ordered list
<code></code>	Defines a list item

HTML Form and Input

The **<form>** Element: The HTML `<form>` element is used to create an HTML form for user input:

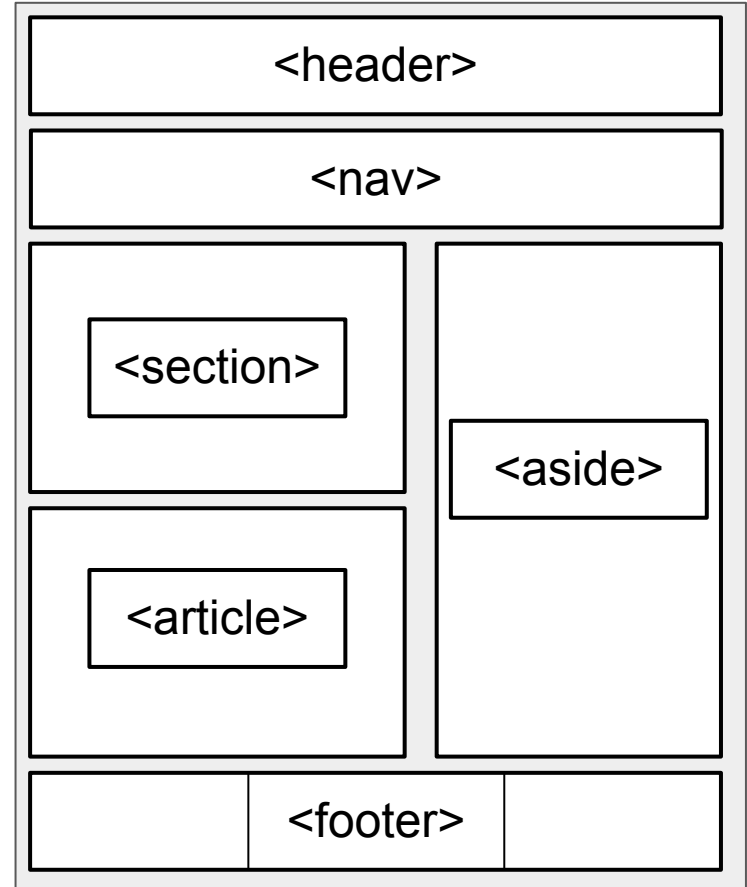
`<form>` form elements `</form>`

The **<input>** Element: The HTML `<input>` element is the most used form element.

Type	Description
<code><input type="text"></code>	Displays a single-line text input field
<code><input type="radio"></code>	Displays a radio button (for selecting one of many choices)
<code><input type="checkbox"></code>	Displays a checkbox (for selecting zero or more of many choices)
<code><input type="submit"></code>	Displays a submit button (for submitting the form)
<code><input type="button"></code>	Displays a clickable button

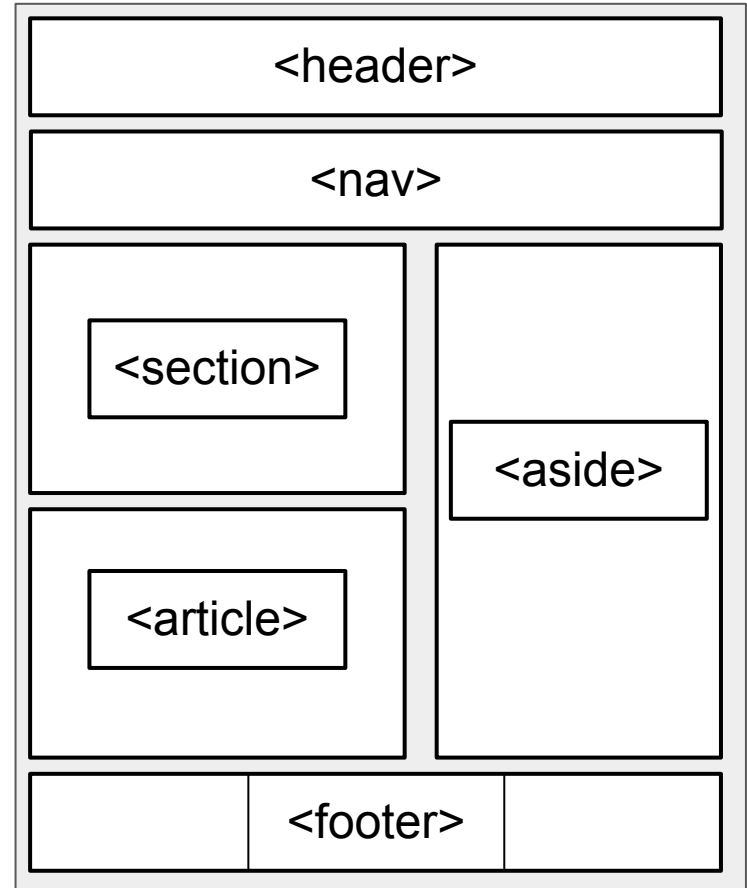
Semantic HTML5 Elements

- ❖ **<header>** The "Intro." Contains introductory content for a page or section (e.g., a logo, navigation menu, or main title).
- ❖ **<nav>** The "Map." Wraps a section of links intended for major navigation (e.g., top menu, table of contents).
- ❖ **<article>** The "Independent Item." Use this for content that stands alone and makes sense even if you remove everything else on the page (e.g., a single blog post, a news story, a forum comment)
- ❖ **<section>** The "Chapter." A thematic grouping of content, usually with a heading. Use this when <div> is too generic but <article> is too specific.



Semantic HTML5 Elements

- ❖ **<aside>** The "Side Note." Use this for content related to the main story but not critical to it (e.g., sidebars, "related posts," or ad banners)
- ❖ **<main>** Represents the dominant content of the <body>. There should only be one unique <main> per page.
- ❖ **<footer>** The "Bottom Info." Contains closing information for a page or section (e.g., copyright, contact info, related links).



Cascading Style Sheets (CSS)

CSS is the language we use to style a Web page.

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files



Class and Id

Class:

A Class is used when you want to style groups of things. You can use the same class name as many times as you want on a single page.

- HTML Attribute: `class="classname"`
- CSS Symbol: `.` (Dot)
- Analogy: "All elements with the class button should be blue."

ID:

An ID is used to identify one specific element. It must be unique on that page. You cannot have two elements with `id="main-logo"`.

- HTML Attribute: `id="idname"`
- CSS Symbol: `#` (Hash/Pound)
- Analogy: "The element with the ID submit-button should be green."

The Anatomy of a CSS Rule

/ Selector Property Value */*

h1 { color: red; }

- Selector: "Who are we talking to?" (The HTML element, e.g., h1).
- Property: "What do we want to change?" (e.g., color, font-size).
- Value: "What is the new setting?" (e.g., red, 20px).

For Class

```
.btn {  
    background-color: blue;  
    color: white;  
}
```

For ID

```
#submit-btn {  
    background-color:  
green;  
    font-weight: bold;  
}
```

For Universal Selector

```
*{  
    background-color:  
green;  
    font-weight: bold;  
}
```

CSS Styles

❖ Inline CSS

- This method puts the style directly inside the HTML tag.
- Syntax: `<tag style="property: value;">`
- Use Case: Quick testing or very specific, one-time overrides (like in emails).
- The Problem: It is impossible to maintain. If you want to change the font of 100 paragraphs, you have to edit 100 tags.
- Example:
- `<h1 style="color: blue; text-align: center;">This is a Blue Headline</h1>`

CSS Styles

- ❖ Internal CSS
 - This method puts the styles in the `<head>` of the specific HTML file.
 - Syntax: Inside `<style>` tags.
 - Use Case: Single-page websites or when you can't upload a separate .css file.
 - The Problem: The styles only work on this one page. If you have a 10-page website, you have to copy-paste this code 10 times.

```
<!DOCTYPE html>
<html>
<head>
  <title>My Page</title>
  <style>
    h1 {
      color: red;
      text-align: center;
    }
    p {
      font-size: 18px;
    }
  </style>
</head>
<body>
  <h1>This is a Red Headline</h1>
</body>
</html>
```

CSS Styles

- ❖ External CSS
 - This method links to a completely separate file (usually style.css).
 - Syntax: A <link> tag in the HTML <head>.
 - Use Case: All real-world websites and applications.
 - The Benefit: You write the code once in style.css, and link it to 50 different HTML pages. If you change the color to green, the whole website updates instantly.

HTML File

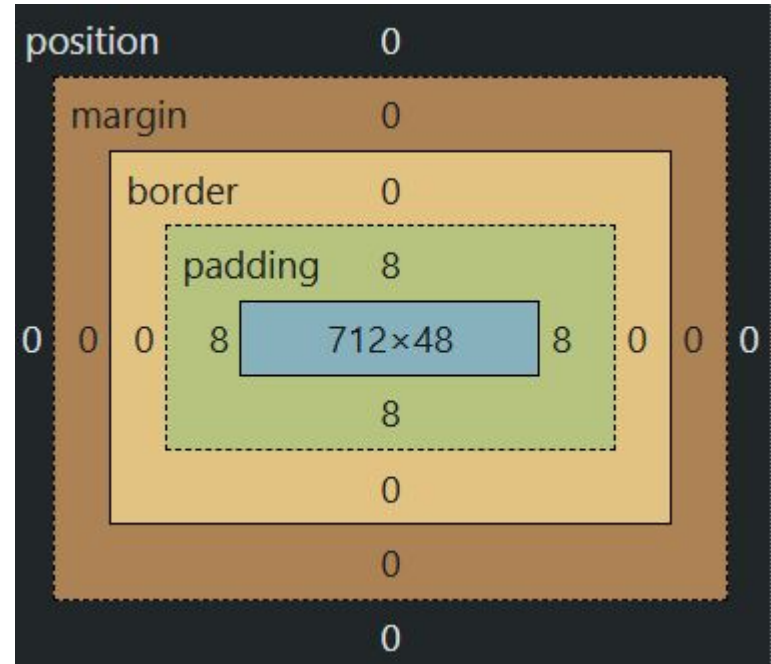
```
<head>  
  <link rel="stylesheet" href="style.css">  
</head>
```

Css File

```
h1 {  
  color: green;  
  text-align: center;  
}
```

CSS: Box Model

- ❖ Content - The content of the box, where text and images appear
- ❖ Padding - Clears an area around the content. The padding is transparent
- ❖ Border - A border that goes around the padding and content
- ❖ Margin - Clears an area outside the border. The margin is transparent



CSS: Flex Box

- ❖ CSS Flexbox is short for the CSS Flexible Box Layout module.
- ❖ Flexbox is a layout model for arranging items (horizontally or vertically) within a container, in a flexible and responsive way.
- ❖ **Flex-direction:** row (Default), and column
- ❖ **Justify-content:**
 - flex-start: Everything to the left (Default).
 - center: Everything in the middle.
 - flex-end: Everything to the right.
 - space-between: Pushes the first item to the far left, the last to the far right, and evens out the space in between
- ❖ **Align-items:**
 - stretch: Stretch to fill the height (Default).
 - center: Center vertically (The most useful one!).
 - flex-start: Stick to the top.

Bootstrap

- ❖ Bootstrap is a free front-end framework for faster and easier web development
- ❖ Bootstrap includes HTML and CSS based design templates for typography, forms, buttons, tables, navigation, modals, image carousels and many other, as well as optional JavaScript plugins
- ❖ Bootstrap also gives you the ability to easily create responsive designs
- ❖ We have different version of bootstrap, in this we will be using bootstrap 5 which was released in 2013.

Why Bootstrap?

- Easy to use
- Responsive features
- Mobile-first approach
- Browser compatibility

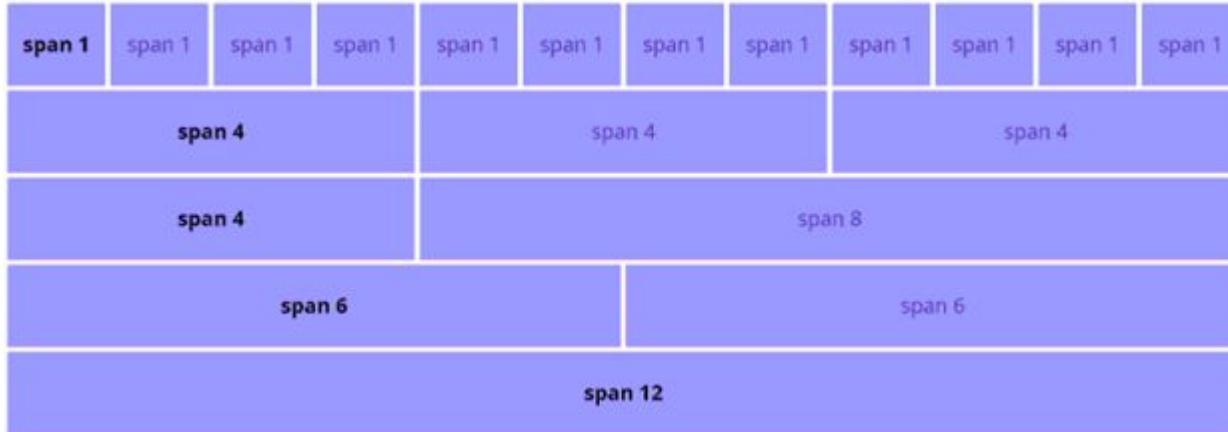
Bootstrap - Container

Containers are used to pad the content inside of them, and there are two container classes available:

1. `.container` (Fixed Width):
 - Keeps content in the middle with white space on the sides.
 - Best for: Blog posts, articles, login forms.
2. `.container-fluid` (Full Width):
 - Stretches across the entire screen (100%).
 - Best for: Navbars, Hero sections, Dashboards.

Bootstrap-Grid System Layout

- ❖ Bootstrap's grid system is built with flexbox and allows up to 12 columns across the page.
- ❖ The grid system is responsive, and the columns will re-arrange automatically depending on the screen size.
- ❖ "Always follow this order: **Container -> Row -> Column.**"



The Responsive Services Page

- .col- (extra small devices - screen width less than 576px)
- .col-sm- (small devices - screen width equal to or greater than 576px)
- .col-md- (medium devices - screen width equal to or greater than 768px)
- .col-lg- (large devices - screen width equal to or greater than 992px)
- .col-xl- (xlarge devices - screen width equal to or greater than 1200px)
- .col-xxl- (xxlarge devices - screen width equal to or greater than 1400px)

Bootstrap: Basic

The Color System:

- ❖ primary = Blue (Main action, like "Post Comment")
- ❖ success = Green (Positive action, like "Save")
- ❖ danger = Red (Negative action, like "Delete")
- ❖ warning = Yellow (Caution, like "Edit")
- ❖ dark / light = Black / White

You combine the property with the color name.

- Background: bg-primary (Blue background)
- Text: text-danger (Red text)
- Button: btn-success (Green button)

A. Property:

- m = Margin (Space outside the box - pushes things away).
- p = Padding (Space inside the box - makes it plump).

B. Side:

- t = Top
- b = Bottom
- s = Start (Left)
- e = End (Right)
- x = X-Axis (Left & Right)
- y = Y-Axis (Top & Bottom)
- (No letter) = All 4 sides.

C. Size (0 to 5):

- 0 = None
- 1 = Small space
- 3 = Medium (Standard)
- 5 = Huge space
- auto = Automatic

Examples for Student:

- mt-5: Margin Top 5 (Push this element down a lot).
- p-3: Padding 3 (Add medium spacing inside the box).
- mx-auto: Margin X Auto (Centers a block element horizontally).

References for html and css

- ❖ <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/number>
- ❖ <https://www.w3schools.com/>
- ❖ <https://getbootstrap.com/docs/5.3/getting-started/introduction/>